
Placement, Not Coverage: Position-Aware CRF Emissions for Proteolytic Segmentation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Predicting where proteases cut a precursor protein is a typed segmentation problem, and it sits under peptide therapeutics, vaccine design and proteomics. Its benchmark counts a segment as correct when both of its ends land within ± 3 residues of a true one. Segments are only 5 to 50 residues long, so that window rewards finding segments over placing them, and a model that places them better gains almost nothing for it. We target placement. The strongest model for this formulation decodes with a conditional random field that distinguishes a segment’s start, interior and end but feeds all of those states one emission per label. We add a zero-initialized boundary head that gives those states position-specific evidence, and a lightweight adapter re-projecting the frozen protein language model embedding before the encoder. Under the 5×4 nested cross-validation the baseline uses, on a rebuilt UniProtKB/Swiss-Prot 2026 dataset, they add 0.054 segment F1 over a 0.576 base and reach 0.666 on ESM-C 6B. What they buy is mostly placement rather than coverage: with detection held fixed, the share of boundaries landing on the annotated residue rises from 0.62 to 0.70 on all five outer folds. The two act on different errors, the head a precision effect and the adapter a recall effect, and the head is worth two and a half times more on the wider embedding, so capacity the base decoder cannot use is capacity the additions can. We also quantify why effects of this size are hard to measure here at all, since homology-aware folds are not exchangeable and their spread exceeds either addition alone.

1 Introduction

Proteomic databases record intact sequences, not what proteases leave behind, so the products of cleavage have to be predicted. A multi-epitope vaccine construct can then be checked at the design stage for the fragments it will yield in vivo, and since proteolysis is itself disrupted in neurodegenerative, oncological and endocrine disease, expected and observed cleavage patterns can be compared between healthy and diseased tissue.

The benchmark for this task rewards detection, not localization. A predicted segment counts as correct if both of its endpoints fall within ± 3 residues of a ground-truth segment of the same type, and a single F1 score at this tolerance is reported (Figure 1). Segments are only 5–50 residues long, so on a short peptide this tolerance is comparable to the length of the segment itself. A model can therefore raise its F1 simply by finding more segments while its boundaries stay imprecise, and conversely it can place boundaries much more precisely and gain almost nothing, because predictions that close already counted as correct. The second case is our concern: the metric cannot see the improvement we are after.

We therefore make boundary localization the target of both the model and the evaluation. Our baseline is DeepPeptide [Teufel et al., 2023b], the strongest general-purpose model for this formulation. It has three parts: a frozen protein language model (pLM), a CNN-BiLSTM encoder, and a linear-

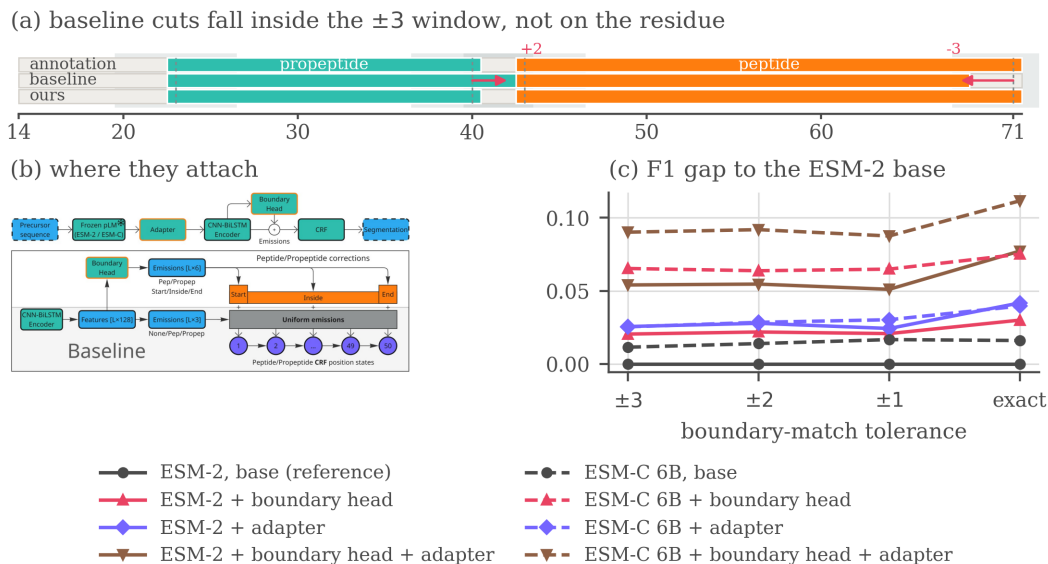


Figure 1: The problem, the two additions and what they buy. (a) One held-out precursor, with the ± 3 acceptance window shaded around each annotated cleavage site. (b) The architecture, with the two additions marked on it. (c) The F1 gap to the ESM-2 base at each tolerance, the base itself drawn flat at zero as the reference every other line is measured against. A curve is flat only if it is a parallel translation of the baseline, and none of the six carrying an addition is. Corrected matcher throughout, means over the five outer folds.

chain conditional random field (CRF). The CRF has separate states for the start, the interior, and the end of a segment, so any valid path must visit them in that order. The decoder is therefore boundary-aware by design. The encoder, however, gives it nothing to work with: every state of the same segment type receives the same emission score. Closing that gap is the natural way to sharpen boundaries rather than merely find more segments. We close it with two small, independent additions and evaluate them under 5×4 nested cross-validation on the DeepPeptide dataset rebuilt from the 2026 UniProtKB/Swiss-Prot release (Sections 4.1 and 5).

Our contributions are the two blocks, the decomposition of what each buys under nested cross-validation, and a quantified account of why this benchmark resists a cleaner protocol: homology-aware folds are not exchangeable, their spread exceeds either addition measured on its own, and averaging it away costs roughly 200 GPU-hours per configuration.

2 Related work

Most predictors of proteolytic cleavage are specific by construction. One line covers a fixed set of proteases with known recognition motifs, one enzyme or family at a time [Duckert et al., 2004, Song et al., 2012, Li et al., 2023, 2020a,b], a second scores cleavage near basic residues for one product class, usually neuropeptides [Southey et al., 2006, Wang et al., 2024], and neither answers what an organism’s full complement of proteases does to a precursor. General-purpose models train a compact head on frozen pLM embeddings [Lin et al., 2023, ESM Team, 2024] but mostly score peptides already excised [Du et al., 2024, Zhu et al., 2025] rather than locating them, and PeptideLocator [Mooney et al., 2013] scores each residue rather than segmenting. DeepPeptide [Teufel et al., 2023b] is, to our knowledge, the only model that segments a precursor into typed peptide and propeptide spans, and it established the homology-partitioned benchmark used here [Teufel et al., 2023a], so we take both its architecture and its data pipeline as our starting point (Appendix A).

3 Method

The base architecture. DeepPeptide passes a frozen ESM-2 embedding through a CNN-BiLSTM stack and decodes it with a linear-chain CRF [Lafferty et al., 2001] that expands each of

{Peptide, Propeptide} into a chain of up to 50 position states, 101 in all, so a legal path can only realize a contiguous segment of admissible length. The encoder computes one emission per label and shares it across all 50 states of that label: the decoder is boundary-aware, the features feeding it are not. The two additions close that gap from opposite ends of the pipeline (Figure 1b, drawn at reading size in Appendix B). Other embedding sources and architectural modifications were screened as well, with verdicts in Appendix F, and Appendix E sets out how the configuration we train relates to the published one.

Boundary head. A small feed-forward block (LayerNorm, Linear, GELU, Linear, hidden size 64) is applied to the contextual per-residue features. For each position and each segment type it emits three numbers, scoring how much that position looks like a segment start, an interior position, or a segment end, and these are added to the emissions of the corresponding position-fixed CRF states. The final linear layer is initialized to zero, so training begins at exactly the base model and the head only learns corrections that lower the loss. It costs 4,678 parameters on a trainable stack of 224,710.

Adapter. The second addition leaves the decoder alone and acts on the input. Before the per-residue embedding reaches the CNN-BiLSTM it passes through LayerNorm, dropout, Linear, GELU, dropout and a second LayerNorm, which reduces its width from 1280 to 256 for ESM-2. A pLM embedding is trained on masked-residue recovery rather than cleavage-site localization, and its feature distribution suits neither this task nor the small head that consumes it, so a trainable re-projection on top of a frozen embedding is the standard remedy. It is the more expensive of the two, a net +232,704 parameters.

4 Data and evaluation

4.1 Dataset

A precursor $x = (a_1, \dots, a_L)$ is labelled per residue with $y_t \in \{\text{None, Peptide, Propeptide}\}$ and contiguous runs of a label form typed segments, so the task is to recover which stretches of a precursor become mature peptides and which become propeptides that are excised and discarded. DeepPeptide built its dataset from PEPTIDE and PROPEP annotations in the 2022 Swiss-Prot release. We rebuilt it with the same pipeline on the 2026 release [UniProt Consortium, 2025]. The collection grows from 8,449 proteins to 9,619 (Table 2 in Appendix C), of which 8,897 both carry ESM-2 embeddings and enter the five folds used here. Folds come from GraphPart [Teufel et al., 2023a] at a 30% pairwise-identity ceiling, balanced by cleavage-motif class. Segment-length filtering, motif balancing and the full composition of the rebuild are given in Appendix C.

4.2 Evaluation criterion

Peptides and propeptides are matched separately. A prediction is a true positive when its start and its end both fall within $\pm\tau$ residues of a true segment of the same type. Unmatched predictions are false positives and unmatched true segments false negatives. Following the original evaluation we keep $\tau = 3$ as the headline setting, since annotated sites carry a few residues of experimental uncertainty. Sweeping τ down to zero then asks a second question, not whether a peptide was found but how precisely its ends were placed. The reference implementation of this criterion carries an upstream variable-shadowing bug that marks the wrong true segment as matched, so everything reported here is re-scored with a corrected matcher (Appendix D).

Agreement with the published baseline. DeepPeptide reports precision 0.68 and recall 0.49 at ± 3 , both segment types together, averaged over the twenty models of its nested cross-validation on the 2022 data, which implies an F1 of 0.570. Running the base architecture on that release under the same protocol scores 0.574 ± 0.069 with the original matcher and 0.590 ± 0.064 with the corrected one of Appendix D. The F1 reproduces, the operating point does not: our original-matcher precision is 0.614 and recall 0.541. The published system tunes its hyperparameters per outer fold, while we hold one configuration fixed across all 20 cells so that every row of Table 1 is measured under the same setting (Appendix E). Under the corrected matcher throughout, the 2026 rebuild gives 0.576 ± 0.029 against that 0.590 ± 0.064 , so it reads 0.014 lower by a margin smaller than either run’s spread across folds.

Table 1: 5×4 nested cross-validation with the corrected matcher. Each entry is the mean over the five outer folds, with the standard deviation across them as a subscript. Precision and recall are at the headline ± 3 tolerance, and the four F1 columns tighten it to an exact residue. *Growth* is how much a row’s F1 advantage over the ESM-2 base widens between ± 3 and an exact match.

Additions	at ± 3		F1 by tolerance				growth
	P	R	± 3	± 2	± 1	exact	
<i>ESM-2</i>							
none	0.621 $\pm$.025	0.539 $\pm$.045	0.576 $\pm$.029	0.544 $\pm$.027	0.467 $\pm$.020	0.354 $\pm$.010	—
boundary head	0.673 $\pm$.029	0.538 $\pm$.036	0.597 $\pm$.026	0.566 $\pm$.024	0.488 $\pm$.015	0.384 $\pm$.015	+0.010 $\pm$.011
adapter	0.628 $\pm$.019	0.579 $\pm$.040	0.602 $\pm$.026	0.572 $\pm$.024	0.491 $\pm$.011	0.396 $\pm$.020	+0.016 $\pm$.009
both	0.667 $\pm$.032	0.598 $\pm$.026	0.630 $\pm$.021	0.599 $\pm$.023	0.518 $\pm$.016	0.432 $\pm$.013	+0.023 $\pm$.019
<i>ESM-C 6B</i>							
none	0.620 $\pm$.017	0.560 $\pm$.027	0.588 $\pm$.016	0.558 $\pm$.013	0.483 $\pm$.007	0.371 $\pm$.013	+0.005 $\pm$.016
boundary head	0.731 $\pm$.010	0.572 $\pm$.030	0.641 $\pm$.022	0.608 $\pm$.026	0.532 $\pm$.035	0.429 $\pm$.030	+0.010 $\pm$.017
adapter	0.598 $\pm$.012	0.607 $\pm$.031	0.602 $\pm$.017	0.573 $\pm$.019	0.497 $\pm$.016	0.394 $\pm$.031	+0.014 $\pm$.023
both	0.690 $\pm$.013	0.646 $\pm$.033	0.666 $\pm$.018	0.636 $\pm$.015	0.554 $\pm$.016	0.466 $\pm$.021	+0.021 $\pm$.023

5 Results

We ran the experiments in two stages.

Screening. We first split the data into seven GraphPart folds with fixed roles: four for training, one for epoch selection, and two held out, one for comparing architectures and one intended as a sealed test, which the screening numbers pool. Against that split we screened roughly a dozen modifications, with the protocol, the verdict table and the per-candidate figures in Appendix F, among them the segment-type trade-offs and data-scaling curves of Figures 6 and 7.

The folds of that split are not interchangeable (Figure 4 in Appendix F), and under one assignment of roles several modifications changed the *sign* of their measured effect between the two held-out folds. A single draw of this kind cannot resolve an effect of 0.02–0.03, so none of those numbers is reported as a finding. It can, however, separate the consistently unhelpful from the promising, and two cleared that bar: the boundary head and the adapter sat at or above the base throughout.

Confirmation. We then re-evaluated the two survivors with the 5×4 nested cross-validation DeepPeptide itself uses, on ESM-2 and on ESM-C 6B, so that the embedding is a factor of the design and not a third candidate. For each of five outer folds we train four models on three of the remaining folds and validate on the fourth, then test on the outer fold, which they never saw. An outer fold’s score is the mean over its four models, the estimate is the mean over the five outer folds, and the reported spread is the standard deviation across outer folds rather than across all 20 cells, since four models sharing an outer fold share a test set. Because selection happened at the screening stage, what follows is a test of two pre-specified hypotheses. Table 1 reports the resulting grid, each addition alone and both together, on both embeddings.¹

The two additions act on different errors. On ESM-2 each improves F1 by a similar amount alone and together they give 0.054, against 0.079 on ESM-C, close to the sum of the parts in both cases. Splitting the ESM-2 rows into precision and recall shows the two are not competing for the same errors. The head is almost purely a precision effect, +0.052 against -0.001 of recall, the adapter mostly a recall effect, +0.040 against +0.007, and applied together they recover both, +0.046 and +0.059.

Tightening the tolerance. Sweeping τ from ± 3 to an exact match, absolute F1 falls steeply for every model, the strongest configuration from 0.666 to 0.466. A model that is better overall retains a different fraction of its ± 3 score even when no boundary is placed better, so we compare absolute gaps rather than ratios. Every gap an addition opens widens as the tolerance tightens, the largest step in each case being the last, from ± 1 to exact, and only the embedding swap on its own does not grow. The *growth* column of Table 1 carries that as a direction rather than as six resolved effects: all

¹Implementation, the configuration behind every row, the per-cell scores every table is aggregated from and a script that rebuilds them: <https://anonymous.4open.science/r/cleavage-site-segmentation-5851>

147 six are positive, on four or five outer folds of five each, but four of the six spreads reach zero. What
148 follows is what the claim rests on.

149 A widening gap is not proof of better localization, since a model that finds more segments gains
150 ground at every tolerance. To hold detection fixed we compared each variant against the base on the
151 true segments both localize within ± 3 , cell by cell. On that set the share of boundaries placed on
152 the exact residue is 0.630 for the base against 0.652 for the boundary head, 0.619 against 0.668 for
153 the adapter and 0.621 against 0.698 for both, each positive on all five outer folds. A gate is itself a
154 choice, so Appendix H asks the same question with no window at all, and there the two blocks part
155 company. When a hit needs only a single residue of overlap the head covers less than the base, recall
156 -0.035 and below it on four folds of five, while gaining precision, and the adapter covers more,
157 $+0.024$ on four of five, while losing it, so their F1 there barely moves, -0.013 and $+0.006$. The
158 head's gain is placement alone, the adapter buys placement and coverage together, and a ± 3 window
159 records the mixture as recall. Scored at individual cleavage sites the head's gain lies entirely on the
160 C-terminal side, $+0.028 \pm 0.006$ on five folds of five against nothing on the N-side.

161 **Capacity the base decoder cannot use.** The base architecture on ESM-C 6B reaches $0.588 \pm$
162 0.016 against 0.576 ± 0.029 on ESM-2, a paired difference of $+0.012 \pm 0.019$ over the five outer
163 folds, so an embedding twice as wide buys no confirmed improvement on its own while either
164 addition on the narrower one does. What they are worth there depends on which one. Paired by
165 outer fold against its own base, the head is worth two and a half times as much on ESM-C 6B as
166 on ESM-2, the adapter about half as much, and the two together more. All six effects are positive
167 on five folds of five, and only the ESM-C adapter's spread reaches zero. The adapter re-projects
168 an embedding trained for masked-residue recovery, and the less mismatched it is the less there is
169 to re-project, whereas the head supplies position-specific evidence and a richer embedding carries
170 more of it. Read the other way, the swap is worth $+0.012$ F1 under the base architecture and $+0.036$
171 under both additions, a split Appendix H reproduces on the protein-cells common to all runs.

172 6 Conclusion and limitations

173 A zero-initialized boundary head at the decoder and a lightweight adapter at the input improve
174 segment F1 by $+0.054$ together on ESM-2 and by $+0.079$ on ESM-C 6B. They combine because
175 they act on different errors, the head suppressing spurious segments and the adapter finding true
176 ones the base misses, and both place boundaries more precisely, so their advantage widens as the
177 tolerance tightens. The evaluation carries a second message: homology-aware partitioning leaves
178 the folds taxonomically non-exchangeable, and their spread exceeds either addition measured on its
179 own.

180 **The folds stay unequal.** Averaging over folds does not make them comparable, it only stops one of
181 them deciding the result. GraphPart keeps homologs together, and taxa *are* homology clusters: 313
182 of 714 *Conus* sequences fall in fold 1 against 16 in fold 2, all 293 *Cyriopagopus* avoid fold 0 entirely,
183 and the folds differ in size by a factor of two (Table 3 in Appendix C). The base architecture's outer-
184 fold scores span 0.074 (Figure 11 and Table 5 in Appendix G), three and a half times the $+0.021$
185 effect the same experiment resolves once averaged over them, and what the folds differ in is segment
186 length, on which quality is far from flat (Figure 12 in Appendix G).

187 **What the protocol does not settle.** Nothing was selected on outer-fold scores inside confirmation,
188 since every cell of the 2×2 is reported, but the screening that chose the candidates (Appendix F)
189 ran on the same proteins the folds are drawn from, and sealing that would take a third held-out level.
190 One cell costs about 10 GPU-hours and the full grid roughly 1,600, so a third level multiplies that
191 while cutting the training fraction from $3/5$ to $2/5$ of an already small dataset. The spread covers
192 fold composition and not seed variation, and the base row is untuned rather than the tuned published
193 system (Appendix E).

194 References

195 José Juan Almagro Armenteros, Marco Salvatore, Olof Emanuelsson, Ole Winther, Gunnar von
196 Heijne, Arne Elofsson, and Henrik Nielsen. Detecting sequence signals in targeting peptides using
197 deep learning. *Life Science Alliance*, 2(5):e201900429, 2019. doi: 10.26508/lsa.201900429.

198 Zhenjiao Du, Xingjian Ding, William Hsu, Arslan Munir, Yixiang Xu, and Yonghui Li. pLM4ACE:
199 a protein language model based predictor for antihypertensive peptide screening. *Food Chemistry*,
200 431:137162, 2024. doi: 10.1016/j.foodchem.2023.137162.

201 Peter Duckert, Søren Brunak, and Nikolaj Blom. Prediction of proprotein convertase cleavage sites.
202 *Protein Engineering, Design and Selection*, 17(1):107–112, 2004. doi: 10.1093/protein/gzh013.

203 Ahmed Elnaggar, Hazem Essam, Wafaa Salah-Eldin, Walid Moustafa, Mohamed Elkerdawy, Char-
204 lotte Rochereau, and Burkhard Rost. Ankh: Optimized protein language model unlocks general-
205 purpose modelling, 2023. URL <https://arxiv.org/abs/2301.06568>.

206 ESM Team. Esm cambrian: Revealing the mysteries of proteins with unsupervised learning. <https://www.evolutionaryscale.ai/blog/esm-cambrian>, December 2024.
207

208 Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert
209 Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat,
210 Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra,
211 Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido,
212 and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*,
213 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.

214 Michael Heinzinger, Konstantin Weissenow, Joaquin Gomez Sanchez, Adrian Henkel, Milot
215 Mirdita, Martin Steinegger, and Burkhard Rost. Bilingual language model for protein sequence
216 and structure. *NAR Genomics and Bioinformatics*, 6(4):lqae150, 2024. doi: 10.1093/nargab/
217 lqae150.

218 John D. Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Prob-
219 abilistic models for segmenting and labeling sequence data. In *Proceedings of the Eighteenth*
220 *International Conference on Machine Learning (ICML)*, pages 282–289, 2001.

221 Fuyi Li, Jinxiang Chen, André Leier, Tatiana Marquez-Lago, Quanzhong Liu, Yanze Wang, Jerico
222 Revote, A. Ian Smith, Tatsuya Akutsu, Geoffrey I. Webb, Lukasz Kurgan, and Jiangning Song.
223 DeepCleave: a deep learning predictor for caspase and matrix metalloprotease substrates and
224 cleavage sites. *Bioinformatics*, 36(4):1057–1065, 2020a. doi: 10.1093/bioinformatics/btz721.

225 Fuyi Li, André Leier, Quanzhong Liu, Yanan Wang, Dongxu Xiang, Tatsuya Akutsu, Geoffrey I.
226 Webb, A. Ian Smith, Tatiana Marquez-Lago, Jian Li, and Jiangning Song. Procleave: predict-
227 ing protease-specific substrate cleavage sites by combining sequence and structural information.
228 *Genomics, Proteomics & Bioinformatics*, 18(1):52–64, 2020b. doi: 10.1016/j.gpb.2019.08.002.

229 Fuyi Li, Cong Wang, Xudong Guo, Tatsuya Akutsu, Geoffrey I. Webb, Lachlan J. M. Coin, Lukasz
230 Kurgan, and Jiangning Song. ProsperousPlus: a one-stop and comprehensive platform for ac-
231 curate protease-specific substrate cleavage prediction and machine-learning model construction.
232 *Briefings in Bioinformatics*, 24(6):bbad372, 2023. doi: 10.1093/bib/bbad372.

233 Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin,
234 Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom
235 Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level
236 protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/
237 science.ade2574.

238 Catherine Mooney, Niall J. Haslam, Gianluca Pollastri, and Denis C. Shields. Towards the im-
239 proved discovery and design of functional peptides: common features of diverse classes permit
240 generalized prediction of bioactivity. *PLoS ONE*, 7(10):e45012, 2012. doi: 10.1371/journal.pone.
241 0045012.

242 Catherine Mooney, Niall J. Haslam, Thérèse A. Holton, Gianluca Pollastri, and Denis C. Shields.
243 PeptideLocator: prediction of bioactive peptides in protein sequences. *Bioinformatics*, 29(9):
244 1120–1126, 2013. doi: 10.1093/bioinformatics/btt103.

245 Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo,
246 Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function
247 emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the*
248 *National Academy of Sciences*, 118(15):e2016239118, 2021.

- 249 Jiangning Song, Hao Tan, Andrew J. Perry, Tatsuya Akutsu, Geoffrey I. Webb, James C. Whisstock,
250 and Robert N. Pike. PROSPER: an integrated feature-based tool for predicting protease substrate
251 cleavage sites. *PLoS ONE*, 7(11):e50300, 2012. doi: 10.1371/journal.pone.0050300.
- 252 Jiangning Song, Fuyi Li, André Leier, Tatiana T. Marquez-Lago, Tatsuya Akutsu, Gholamreza Haf-
253 fari, Kuo-Chen Chou, Geoffrey I. Webb, and Robert N. Pike. PROSPERous: high-throughput
254 prediction of substrate cleavage sites for 90 proteases with improved accuracy. *Bioinformatics*,
255 34(4):684–687, 2018. doi: 10.1093/bioinformatics/btx670.
- 256 Bruce R. Southey, Andinet Amare, Tyler A. Zimmerman, Sandra L. Rodriguez-Zas, and Jonathan V.
257 Sweedler. NeuroPred: a tool to predict cleavage sites in neuropeptide precursors and provide the
258 masses of the resulting peptides. *Nucleic Acids Research*, 34(Web Server issue):W267–W272,
259 2006. doi: 10.1093/nar/gkl161.
- 260 Felix Teufel, José Juan Almagro Armenteros, Alexander Rosenberg Johansen, Magnús Halldór Gís-
261 lason, Silas Irby Pihl, Konstantinos D. Tsirigos, Ole Winther, Søren Brunak, Gunnar von Heijne,
262 and Henrik Nielsen. SignalP 6.0 predicts all five types of signal peptides using protein language
263 models. *Nature Biotechnology*, 40(7):1023–1025, 2022. doi: 10.1038/s41587-021-01156-3.
- 264 Felix Teufel, Magnús Halldór Gíslason, José Juan Almagro Armenteros, Alexander Rosenberg Jo-
265 hansen, Ole Winther, and Henrik Nielsen. GraphPart: homology partitioning for biological se-
266 quence analysis. *NAR Genomics and Bioinformatics*, 5(4):lqad088, 2023a. doi: 10.1093/nargab/
267 lqad088.
- 268 Felix Teufel, Jan Christian Refsgaard, Christian Toft Madsen, Carsten Stahlhut, Mads Grønborg, Ole
269 Winther, and Dennis Madsen. Deeppeptide predicts cleaved peptides in proteins using conditional
270 random fields. *Bioinformatics*, 39(10):btad616, 10 2023b. ISSN 1367-4811. doi: 10.1093/
271 bioinformatics/btad616. URL <https://doi.org/10.1093/bioinformatics/btad616>.
- 272 UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Re-*
273 *search*, 53(D1):D609–D617, 2025. doi: 10.1093/nar/gkae1010. URL <https://academic.oup.com/nar/article/53/D1/D609/7902999>.
- 275 Michel van Kempen, Stephanie S. Kim, Charlotte Tumescheit, Milot Mirdita, Jeongjae Lee,
276 Cameron L. M. Gilchrist, Johannes Söding, and Martin Steinegger. Fast and accurate pro-
277 tein structure search with Foldseek. *Nature Biotechnology*, 42:243–246, 2024. doi: 10.1038/
278 s41587-023-01773-0.
- 279 Lei Wang, Chen Huang, Mingxia Wang, Zhidong Xue, and Yan Wang. NeuroPred-PLM: an inter-
280 pretable and robust model for neuropeptide prediction by protein language model. *Briefings in*
281 *Bioinformatics*, 24(2):bbad077, 2023. doi: 10.1093/bib/bbad077.
- 282 Lei Wang, Zilu Zeng, Zhidong Xue, and Yan Wang. DeepNeuropePred: a robust and universal tool
283 to predict cleavage sites from neuropeptide precursors by protein language model. *Computational*
284 *and Structural Biotechnology Journal*, 23:309–315, 2024. doi: 10.1016/j.csbj.2023.12.004.
- 285 Ming Zhang, Jianren Zhou, Xiaohua Wang, Xun Wang, and Fang Ge. DeepBP: ensemble deep
286 learning strategy for bioactive peptide prediction. *BMC Bioinformatics*, 25(1):352, 2024. doi:
287 10.1186/s12859-024-05974-5.
- 288 Lun Zhu, Hao Sun, and Sen Yang. BPFun: a deep learning framework for bioactive peptide function
289 prediction using multi-label strategy by transformer-driven and sequence rich intrinsic informa-
290 tion. *BMC Bioinformatics*, 26(1):187, 2025. doi: 10.1186/s12859-025-06190-5.

291 Appendix

292 A Extended review of prior work

293 Methods for predicting proteolytic peptides fall into three broad classes.

294 **Protease-specific models.** These predict cleavage only for a limited set of enzymes, most of which
295 have a known recognition motif. ProP [Duckert et al., 2004] targets cleavage by the PACE/PC family
296 in animals and plants. PROSPER [Song et al., 2012] and its successor PROSPERous [Song et al.,
297 2018] predict cleavage for one chosen enzyme at a time, as does the later ProsperousPlus [Li et al.,
298 2023]. DeepCleave [Li et al., 2020a] is restricted to caspases and matrix metalloproteinases, and
299 Procleave [Li et al., 2020b] likewise handles one enzyme at a time and additionally requires 3D
300 structural input. The shared limitation of this class is scope: in practice, one is usually interested in
301 the combined effect of all proteases active in an organism, not any single enzyme.

302 **Organism- or peptide-type-specific models.** A separate class targets neuropeptides or other bioac-
303 tive fragments of one product type. NeuroPred [Southey et al., 2006] and the species-agnostic
304 DeepNeuroPred [Wang et al., 2024] only detect cleavage near a handful of basic residues, while
305 NeuroPred-PLM [Wang et al., 2023] does not localize cleavage sites at all but classifies an already-
306 excised sequence. A related group of models finds signal subsequences rather than any bioactive
307 peptide: SignalP [Teufel et al., 2022] and TargetP [Almagro Armenteros et al., 2019] identify the
308 type of signal peptide, which indicates where a protein is trafficked.

309 **General-purpose models on pLM embeddings.** Protein language models (pLMs) are transform-
310 ers trained without supervision on large sequence corpora, typically by masked-residue recovery,
311 and they produce transferable per-residue embeddings: ESM-2 [Lin et al., 2023], the earlier ESM
312 line [Rives et al., 2021], ESM Cambrian [ESM Team, 2024, Hayes et al., 2025], and Ankh [El-
313 naggar et al., 2023]. Compact task-specific heads are then trained on top of these frozen embed-
314 dings. Besides **DeepPeptide** [Teufel et al., 2023b] itself, this recipe underlies DeepNeuroPred,
315 NeuroPred-PLM, and SignalP 6, as well as models that score already-excised peptides rather than
316 finding them: pLM4ACE [Du et al., 2024] predicts ACE-inhibitory activity, and BPFun [Zhu et al.,
317 2025] and DeepBP [Zhang et al., 2024] predict broader bioactivity, building on earlier pre-pLM
318 work such as PeptideRanker [Mooney et al., 2012]. Before pLMs, PeptideLocator [Mooney et al.,
319 2013] addressed a problem close to ours, but its output is a per-residue bioactivity score rather than
320 an explicit segmentation, and it was the main point of comparison in the original DeepPeptide paper.

321 **Baseline architecture: DeepPeptide.** Since we build directly on it throughout this paper (Sec-
322 tion 3), DeepPeptide’s architecture is worth describing in detail rather than treating as a black box.
323 It first encodes a precursor sequence with a frozen pLM (ESM-2), whose per-residue embedding is
324 never updated. This embedding is refined by a convolution, a single-layer bidirectional LSTM and
325 a second convolution. The published description does not fix the widths: they are searched with
326 Optuna inside the inner cross-validation loop. The configuration we train throughout uses 32 con-
327 volutional filters of width 3 and an LSTM hidden size of 64 per direction, giving 224,710 trainable
328 parameters for the base model. These features are consumed by a linear-chain CRF decoder whose
329 state set is extended well beyond the three raw labels {None, Peptide, Propeptide}: each of the
330 two positive labels is expanded into a chain of states long enough to represent segments up to a fixed
331 maximum length (101 states in total for the 50-residue cap used here, Section 4.1), so that a legal
332 path through the CRF can only realize a contiguous segment of valid length, with dedicated states
333 marking its first and last positions. This makes segmentation, rather than independent per-residue
334 classification, the object the model is directly optimized for – and it is also where the gap described
335 in Section 1 lives: every state in this extended set that shares a type receives the same emission from
336 the encoder, regardless of whether it marks the start, interior, or end of a segment.

337 **Baseline dataset for this task.** Beyond its architecture, DeepPeptide also established the data re-
338 source this line of work relies on: precursor sequences from UniProtKB/Swiss-Prot annotated with
339 PEPTIDE and PROPEP feature types, partitioned into homology-aware folds with GraphPart [Teufel
340 et al., 2023a] at a 30% pairwise-identity threshold and additionally balanced by cleavage-flanking
341 motif, then evaluated with the same family of nested cross-validation we adopt (Section 5). This
342 gave the field a standardized, already homology-controlled benchmark rather than an ad hoc collec-
343 tion of previously published peptide lists, and both the curated data and the construction pipeline
344 that produced it are public. Section 4.1 describes what we do with that resource: rebuilding it on a

current UniProtKB/Swiss-Prot release, which adds 1,170 proteins net and refreshes the annotations behind every segment.

Among these, DeepPeptide is, to our knowledge, the only model that poses the task as full segmentation of the precursor into typed peptide and propeptide segments, and it is the strongest available baseline for this formulation. We therefore use its architecture and its dataset-construction methodology as the starting point for this work, rather than treating it as a system to audit: our contribution is an architectural addition to this general recipe (Section 3), evaluated under the same 5×4 nested cross-validation it uses (Section 5) on data rebuilt from a current release (Section 4.1).

B The architecture at reading size

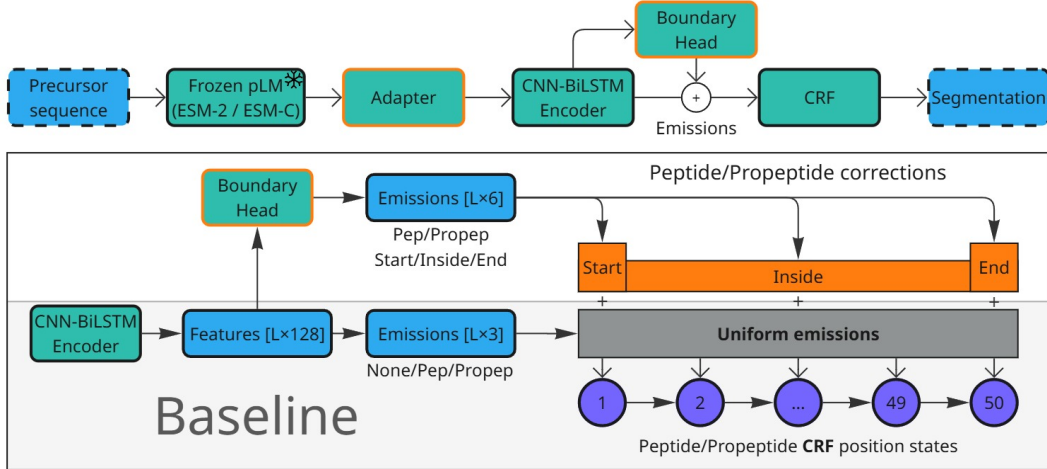


Figure 2: The base architecture and the two additions, at reading size. Figure 1b carries a single-panel version of this diagram, which locates the two blocks rather than documenting them. Top: the pipeline, with the adapter between the frozen pLM and the CNN-BiLSTM encoder and the boundary head reading the encoder’s output. Bottom: what the head adds. The base model computes one emission per label and shares it across all 50 position states of that label, so the CRF’s start, interior and end states are fed identical evidence. The head emits three numbers per position and per segment type and adds them to the states they describe.

C Dataset details

Table 2: Dataset composition: the 2022 release used by the original DeepPeptide against the 2026 release rebuilt here. Counts are before homology partitioning, which leaves 7,623 proteins of the 2022 release and 8,994 of the 2026 one, of which 8,897 also carry an ESM-2 embedding and form the five folds used here.

	2022	2026
Proteins	8449	9619
Peptide segments	6372	7431
Propeptide segments	8211	9140
Median peptide length (residues)	21	20

Two filters from the original pipeline are kept unchanged. Segments shorter than 5 or longer than 50 residues are dropped: the CRF’s state count fixes the upper bound, and a segment shorter than five residues is barely scoreable at a ± 3 tolerance. Folds are then balanced by cleavage-motif class, obtained by k -means ($k = 50$) on ESM-2 embeddings of the four residues flanking each annotated boundary, on top of the 30% pairwise-identity ceiling GraphPart enforces between folds.

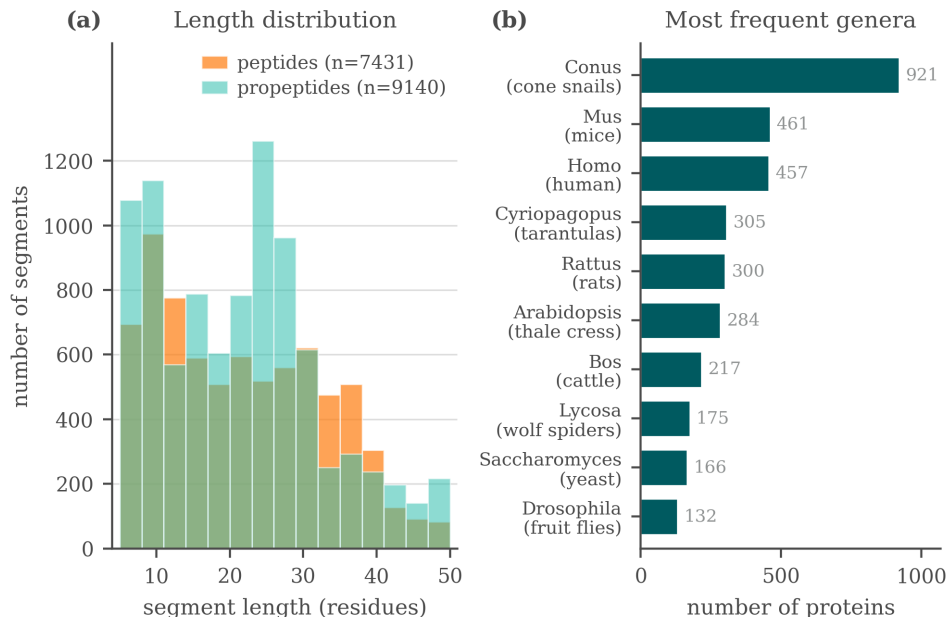


Figure 3: Composition of the rebuilt 2026 dataset: segment length, type, and genus distributions.

Table 3: Proteins per outer fold, and how three frequent genera distribute across them. *Cyriopagopus* and *Lycosa* are spiders, *Conus* a cone snail, and each is a homology cluster that GraphPart is obliged to keep intact. *Homo*, by contrast, is spread evenly (71/81/66/130/84).

	total	f0	f1	f2	f3	f4
All proteins	8897	1558	2572	1263	2025	1479
<i>Conus</i>	714	160	313	16	149	76
<i>Cyriopagopus</i>	293	0	62	45	9	177
<i>Lycosa</i>	163	5	42	10	106	0

D Metric implementation bug

While auditing the DeepPeptide implementation of the matching criterion (Section 4.2), we found a bug in the function that implements it (`get_counts_for_protein`), reproduced below in simplified form:

```

364 for idx, row in true_df.iterrows(): # idx: true-segment index
365     true_start, true_stop = row['start'], row['stop']
366     for idx, row in pred_df.iterrows(): # idx overwritten with pred index
367         pred_start, pred_stop = row['start'], row['stop']
368         ...
369         if start_match and stop_match:
370             true_df.loc[idx, 'matched'] = True # BUG: idx is now the
371             pred_df.loc[idx, 'matched'] = True # predicted index, not the true one
372         break
373 
```

The outer loop iterates over true segments and the inner loop over predicted segments, but both reuse the loop variable `idx`. By the time a match is found, `idx` refers to the predicted segment, so `true_df.loc[idx, 'matched']` marks the wrong true segment. The effect is not large on average (it is partly self-correcting), but it costs a true positive whenever it fires, so both recall and precision come out low, and F1 with them. Across the nested-CV grid the correction raises recall by 0.021 to 0.032 and precision by 0.005 to 0.010. The loss concentrates in proteins where the model predicts more segments than the protein actually has. This bug is present in the upstream DeepPeptide code, not something we introduced.

We did not silently patch this function in place, since that would shift absolute numbers without documenting it. Instead we implemented a separate, corrected matcher and used it to re-score the

385 results in this paper: the earlier single-split runs reported in Appendix F, except two whose model
 386 classes no longer exist in the code and whose checkpoints cannot be rebuilt, and the full 5×4
 387 nested-cross-validation grid in Table 1 (all 160 cells of the eight configurations, and the 20 further
 388 cells of the 2022-release reproduction), by re-running test-partition inference from each cell’s saved
 389 checkpoint on the machine that held them. Every table in the main text uses the corrected matcher,
 390 and both readings of every cell are released alongside the code, with a script that rebuilds the paper’s
 391 tables from them.

392 Re-scoring the grid also recomputed the original matcher on the same decoded segments, so each cell
 393 could be checked against the number already recorded for it. Of the 180 cells, 175 carry a recorded
 394 number, and over 174 of those the two readings agree to 1.2×10^{-3} F1 at worst and to 3×10^{-7} at
 395 the median, so what remains there is inference nondeterminism rather than disagreement between
 396 two deterministic functions of the same segments. One cell disagrees by 2.6×10^{-2} : its saved
 397 checkpoint does not reproduce the F1 its own training run recorded, two independent re-scorings
 398 agree with each other and not with that record, and the re-scored value is the one used here. The
 399 remaining five cells never recorded a number and are scored from their saved segments in the same
 400 way as the rest.

401 Across configurations the correction shifts mean F1 by +0.015 to +0.022, always upward, since
 402 the bug was discarding true positives. On ESM-2 it leaves the ordering of the four configurations
 403 intact, base < boundary head < adapter < combined, before and after correction. That ordering
 404 is not robust and we build no argument on it: the boundary-head/adaptor gap is 0.005 against a
 405 fold-level standard deviation of 0.026, and per outer fold the full ordering holds in four folds of five
 406 after correction and three of five before it. On ESM-C 6B the ordering differs, base < adapter <
 407 boundary head < combined, the two middle configurations swapping places.

408 E Relation to the published DeepPeptide system

409 Our base architecture is DeepPeptide’s architecture, but it is not DeepPeptide’s published model,
 410 and the difference is worth stating because it is the reason Section 4.2 reproduces the published F1
 411 without reproducing the published precision and recall.

412 The released implementation spends the inner loop of its nested cross-validation on a hyperparam-
 413 eter search, and the twenty checkpoints behind the published numbers carry a different winning
 414 configuration for each outer fold: learning rate from 3.3×10^{-4} to 5.5×10^{-3} , dropout from 0.03
 415 to 0.69, convolutional dropout from 0.04 to 0.51, kernel size 3 or 5, 48 to 96 filters, hidden size 32
 416 or 48, and batch size 20 to 90. We hold a single configuration fixed across all twenty cells at the
 417 implementation’s defaults (learning rate 10^{-4} , dropout and convolutional dropout 0.1, kernel size 3,
 418 32 filters, hidden size 64, seed 42), with batch size 48 rather than the default 100, and spend the inner
 419 loop on epoch selection alone. The two added blocks carry settings of their own, chosen once at the
 420 screening stage and then held fixed as well: the adapter projects to 256 channels with a dropout of
 421 0.4, and the boundary head uses a hidden size of 64.

422 Our settings are the implementation’s defaults and they sit outside the range the published search
 423 explored: a learning rate below all five of its ESM-2 winners, fewer convolutional filters than any
 424 of them, and a wider recurrent hidden state than any of them. The base row is therefore a different
 425 model from the published one, not a worse-tuned copy of it, which is what the precision-recall
 426 difference reflects.

427 That choice is deliberate. A factorial is only interpretable if its cells differ in the factor under study
 428 and in nothing else, and per-fold tuning would confound each addition with whatever the search
 429 happened to find for it. Two consequences follow, and they are not symmetric between the two
 430 blocks.

431 The boundary head is protected by its initialization. Its final linear layer starts at zero, so at the
 432 first step the model with the head *is* the base model, and the head’s 4,678 parameters can only
 433 add corrections that lower the loss from there. Whatever setting suits the base is by construction
 434 a legitimate setting for the head, which cannot be handicapped relative to the base it is added to.
 435 The adapter has no such protection. It changes the input width from 1280 to 256 and adds 232,704
 436 parameters, roughly doubling the trainable stack, and a model of that shape may well want a different

learning rate and dropout from the one we hold fixed. Its measured effect, +0.026 on ESM-2 and +0.014 on ESM-C 6B, should therefore be read as a lower bound.

The direction of this bias is the useful part: holding hyperparameters fixed can hide an effect but cannot manufacture one, so every gain in Table 1 is conservative. What it does not license is the reading that an untuned base makes the additions look better than they are. On the 2022 release our untuned base reaches an F1 of 0.574 against the tuned published model’s implied 0.570 (Section 4.2), so the base we improve on is not a weak one.

Two further notes on the reference implementation. Its recall guard tests the wrong denominator, returning zero when no segment is predicted but dividing by the true-segment count that the guard never checked. On this data the branch is unreachable, so the fault is latent rather than active. And its evaluation script builds the ground truth with overlapping annotations kept separate while its training dataset merges them, a discrepancy we did not attempt to reconcile because our scoring rebuilds the ground truth from the labelled sequences directly.

F Screening protocol and verdicts

F.1 Protocol

Data was split into seven GraphPart folds (30% identity threshold, motif-balanced as in Section 4.1), with folds assigned fixed roles: four folds for training, one for validation (epoch selection), one for model selection (comparing architectures), and one originally intended as a fully sealed test fold. This separated architecture selection from evaluation, but the evaluation itself was still a single draw: one random assignment of roles, one pair of held-out folds. Several modifications changed the sign of their measured effect between the two held-out folds. Replacing ESM-2 with ESM-C 6B measured +0.074 on one and −0.011 on the other, ESM-C 6B against ESM-C 600M gave +0.028 and −0.021, and the net effect of the 3Di channel gave +0.015 and −0.018. Effects that kept their sign still moved by a lot: the isolated sequence adapter measured −0.001 on one fold and +0.045 on the other. Figure 4 shows one reason, namely that the folds differ substantially in segment-length composition, and Figure 5 shows why that difference matters: under this protocol the base architecture scores 0.724 on segments of 25–29 residues and 0.230 on those of 45–50, so a fold’s length profile largely sets how hard it is before any model is chosen. The two held-out folds turn out to be complementary in segment-length composition, fold 2 concentrated at 10–24 residues and fold 5 bimodal, so results below pool them (model-select $\cup$ sealed, $\approx 2,300$ proteins) to reduce (but not eliminate) this instability. This pooling is why the “sealed” fold is not, in practice, a held-out test set independent of model selection.

F.2 Summary of tested modifications

Table 4 summarizes the modifications tested under the screening protocol, with the corrected matcher (Appendix D) applied throughout. Two of these rows were carried forward as candidates, the boundary head and the adapter (Section 3), and both were confirmed. The embedding swap was not a candidate but became a factor of the confirmation design, and it is the row this table reads least well: nested cross-validation puts ESM-C 6B at 0.588 ± 0.016 against ESM-2’s 0.576 ± 0.029 (Table 1), overlapping intervals, where this table records +0.03. Everything else below remains at the confidence of a single pooled split.

Note the apparent tension with Table 1: under this earlier protocol, the boundary head on ESM-2 looked like it had no effect, while nested cross-validation resolved a confirmed +0.021 F1 gain for the same modification on the same embedding. We read this as evidence that the single-split protocol lacked the statistical power to resolve an effect of this size, not as a contradiction – and as the clearest illustration of why we do not treat single-split numbers as findings in this paper. The interaction these rows suggest, that the boundary head is worth more on the richer embedding, is not established by them: it is established under nested cross-validation in Section 5, where the same comparison is paired by outer fold and positive on five folds of five. What this table contributes is the earlier and much noisier version of it.

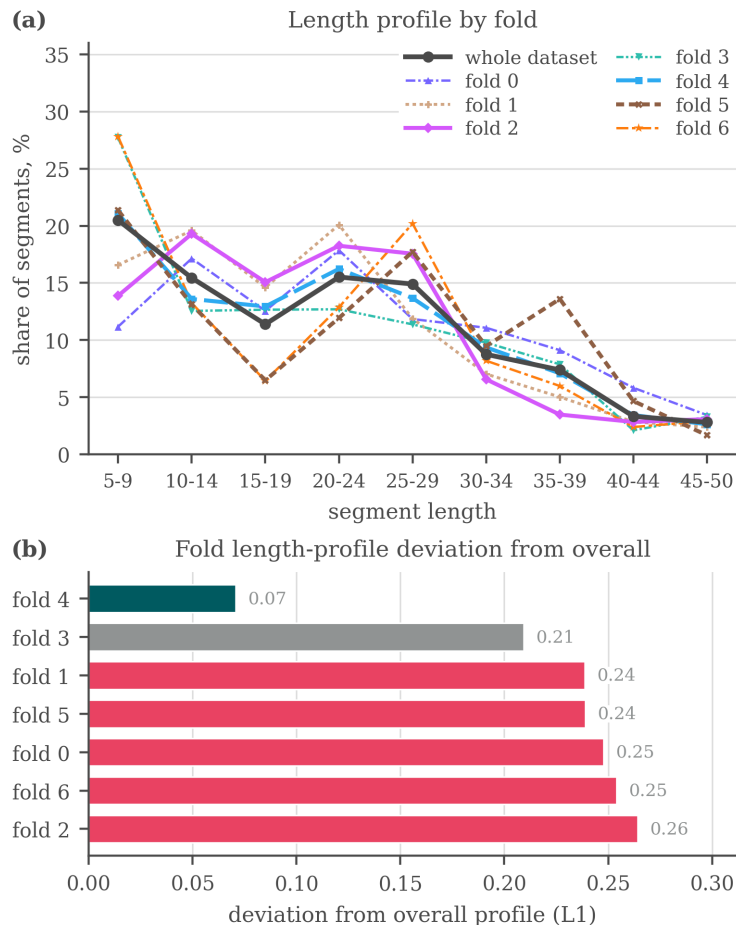


Figure 4: Why the folds of the earlier seven-fold split are not interchangeable: (a) the segment-length profile of each fold against the profile of the whole dataset, (b) the L_1 distance between the two. Fold 4 tracks the overall distribution closely (0.07) while five of the seven folds sit above 0.23. Balancing was done on cleavage motif and homology, not on segment length, so this axis was left free. The same holds for the five-fold split used in the main text, where the imbalance is taxonomic (Table 3).

486 F.3 Additional exploratory modifications

487 A larger set of ideas was tried under the same single-split protocol and did not show a clear, con-
 488 sistent benefit. We list them for completeness and as candidates for future re-testing, without confi-
 489 dence intervals:

- 490 • **Known-peptide dictionary.** An Aho–Corasick index of peptides from the training data
 491 was used to flag substring matches in a query sequence, injected as a bonus on the CRF
 492 state emissions, as a bias on the start, inside and end emissions specifically, fused into
 493 the encoder hidden state, and concatenated with the embedding before the encoder. No
 494 consistent benefit was observed, and the checkpoint for one of the variants can no longer
 495 be loaded.
- 496 • **Structural features.** Features extracted from predicted 3D structures (AlphaFold2 rep-
 497 resentations via the AFToolkit codebase, with several confidence-based filtering variants)
 498 and from the ProstT5 [Heinzinger et al., 2024] structural alphabet (3Di, as used in Fold-
 499 seek [van Kempen et al., 2024]) were tested as additional input channels, and results were
 500 mixed (Table 4).
- 501 • **LoRA fine-tuning.** Low-rank adaptation of the last few pLM layers, instead of fully frozen
 502 embeddings, scored 0.558 and 0.567 against 0.621 for the frozen baseline, all three on the

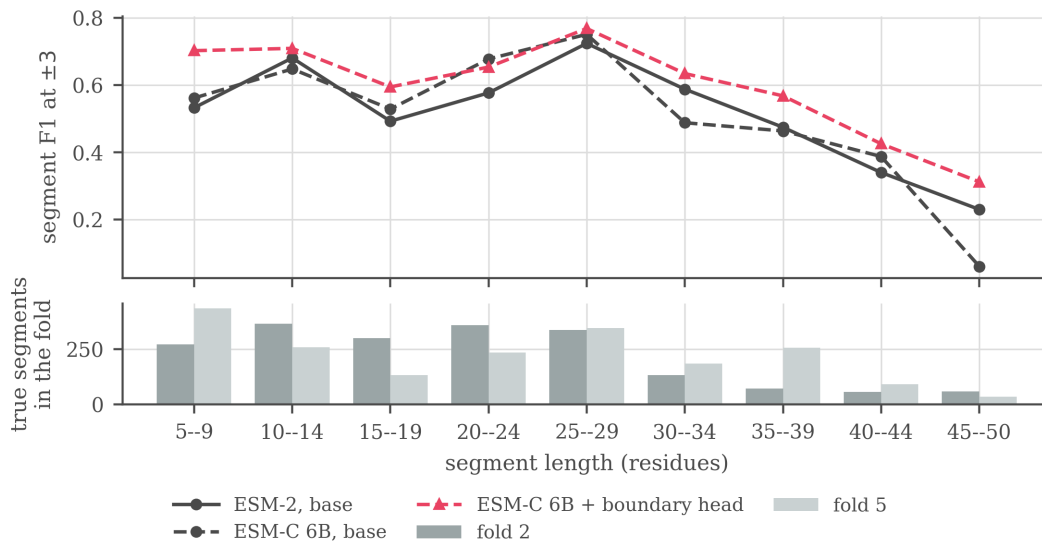


Figure 5: Segment F1 at ± 3 against the length of the true segment under the seven-fold screening split, with the number of true segments each held-out fold contributes to a bin below. Quality depends strongly on length for every model: the base architecture scores 0.724 on segments of 25–29 residues and 0.230 on those of 45–50, and ESM-C 6B falls further over the same range, from 0.751 to 0.061. The two held-out folds are composed differently on that axis, fold 5 carrying 13.0% of its segments at 35–39 residues against 3.6% in fold 2, so an effect measured on one fold is measured on an easier or harder mixture than the same effect on the other. That is one mechanism behind the sign flips reported above. These are screening numbers and are not comparable with the nested cross-validation values in Appendix G.

Table 4: Modifications tested under the screening single-split protocol, with their measured effect on F1 (pooled held-out folds) and an informal verdict. None of these effect sizes should be compared directly to the nested-CV numbers in Table 1.

Modification	Type	Measured effect on F1	Verdict
ESM-C 6B instead of ESM-2	embedding swap	+0.03, unstable across folds	helps
Boundary head on ESM-C 6B	decoder addition	+0.05, +0.07, consistent	helps
Boundary head on ESM-2	decoder addition	≈ 0 (CI includes zero)	no effect
Structural channel (3Di)	extra input	prop. +0.02 / pep. -0.03 , net trade-off	no effect
ESM-C 6B compression 2560 $\rightarrow$ 256	optimization	≈ 0 , 10 $\times$ narrower input and 16 $\times$ fewer trainable parameters	no effect
Sequence adapter on ESM-C 6B (pLM re-projection)	input adapter	+0.022, CI [+0.008, +0.038]	helps
Bond-prediction auxiliary loss (on ESM-C 6B)	extra loss	pep. -0.05 , net -0.015 , neutral on ESM-2	harmful
Telescopic segment CRF	decoder addition	≈ 0 on ESM-2, +0.022 on ESM-C 6B, no CI available	unresolved

2022 release under its own five-fold split rather than the protocol of Appendix F. It also ran for fewer epochs in the same time budget, though we did not measure the cost directly.

- **Projector variants.** Multi-scale and multi-branch projectors between the embedding and the CNN-BiLSTM were tried as alternative re-projection schemes to the adapter of Section 3.
- **Structural-projection width and telescopic CRF.** The width of the structural-feature projection (16/32/48 units) left pooled F1 flat at 0.691, 0.697 and 0.693, the telescopic segment CRF, an alternative decoder formulation that scores whole segments through a relative-

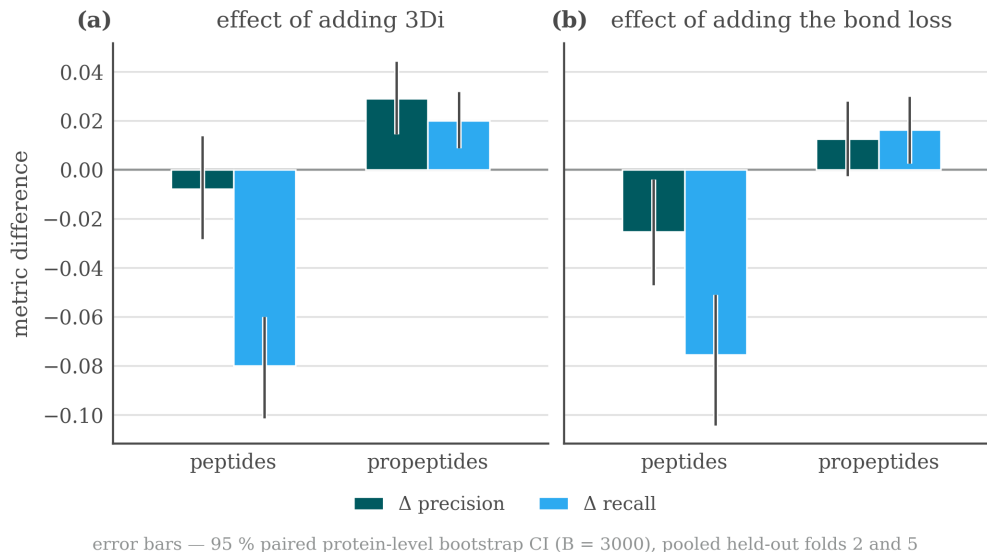


Figure 6: Precision/recall trade-offs by segment type when adding the 3Di channel or the bond-prediction loss, single-split protocol, pooled held-out folds.

position head, came out level with the base decoder on ESM-2 and +0.022 ahead on ESM-C 6B. Per-protein outputs were not retained for either run, so neither number carries a confidence interval and we treat the modification as untested rather than as having no effect.

None of these modifications showed a consistent gain large enough, under the single-split protocol, to justify testing under nested cross-validation ahead of the two reported in the main text. The complete experiment log (including runs not summarized above) is maintained in the project repository rather than reproduced here, since it was collected under a protocol we no longer treat as sufficient evidence on its own.

F.4 What the screening runs looked like

Three figures from the screening stage are worth keeping, each for a different reason, and none of them should be read as confirmed evidence in the sense of Section 5. Figure 8 is the comparison the verdict column of Table 4 summarizes, with the bootstrap intervals that made most of those verdicts unsafe. Figure 9 is the earliest form of the interaction that Section 5 later confirms under nested cross-validation, and shows that the boundary head was already worth more on the wider embedding a protocol ago. Figure 10 asks what recall actually tracks: for both ESM-C models it follows the maximum sequence identity between a held-out peptide and any training segment far more closely than it follows how many training segments the protein's genus contributes, which is the behaviour of a model retrieving near neighbours rather than one that has learned the cleavage grammar. On the ESM-2 baseline the two axes are closer to comparable. This is a single split and we draw no conclusion from it, but it is the observation that would most repay a proper test.

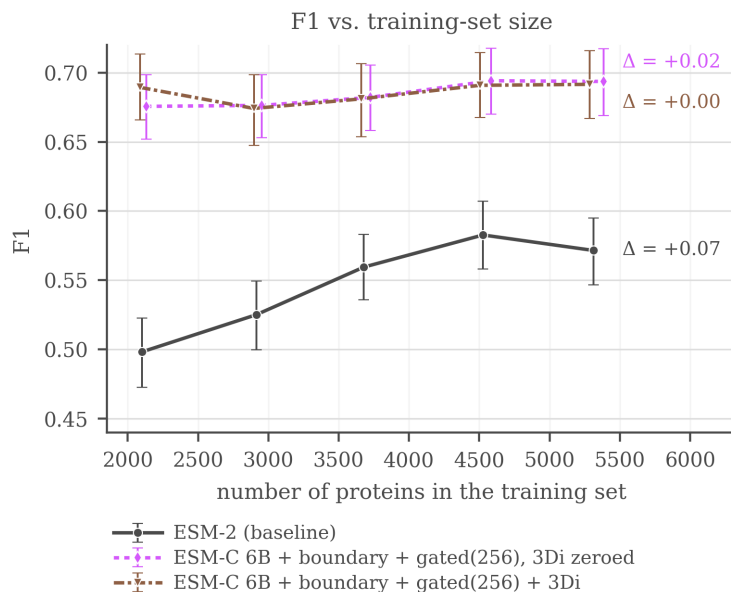
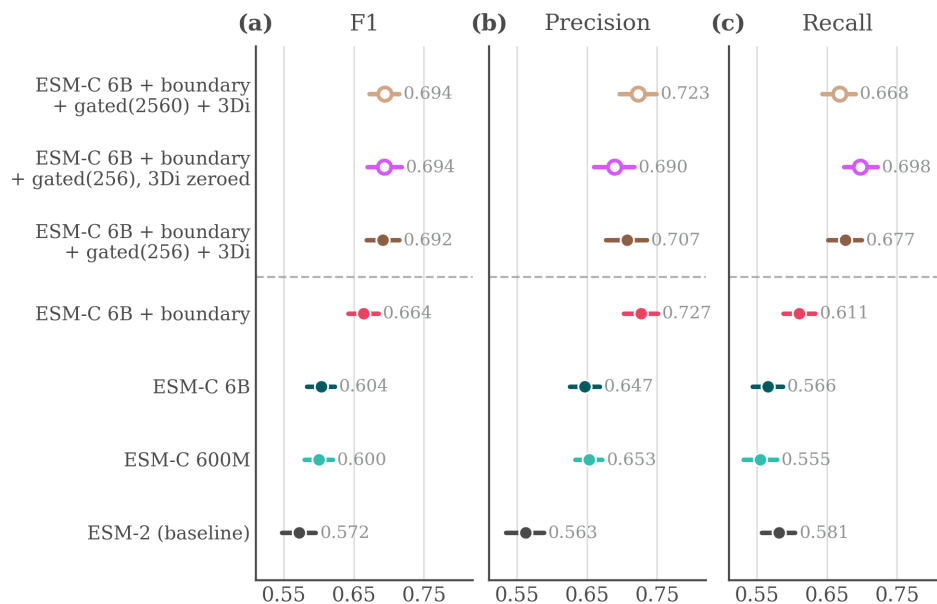


Figure 7: F1 (± 3) against the number of training proteins, single-split protocol. ESM-2 rises from 0.498 at 40% of the training folds to 0.583 at 85%, then falls back to 0.572 at 100%, so it is still gaining over most of the range but not at the last point. The ESM-C 6B curves are flat from the smallest size tested, with movement smaller than the bootstrap interval, so nothing is observed saturating: they simply never climb.



hollow markers — gated-model controls, not rungs of the ladder
dashed rule — gated adapter: +0.022 (isolated by ablation),
significant on average, but the gain is mostly on fold 5

Figure 8: Screening comparison on the pooled held-out folds at ± 3 : F1, precision and recall with bootstrap confidence intervals over 2,322 proteins. The top row is a gated-projector control at full width, not the configuration carried into the main text.

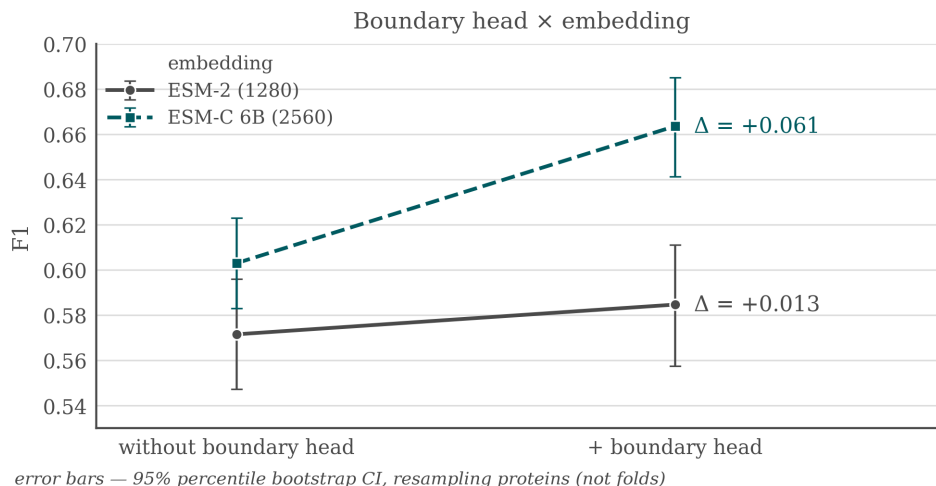


Figure 9: Boundary head × embedding interaction under the screening protocol: F1 gain from adding the boundary head on top of ESM-2 against on top of ESM-C 6B. This is the earlier, unconfirmed version of the interaction that Section 5 measures under nested cross-validation.

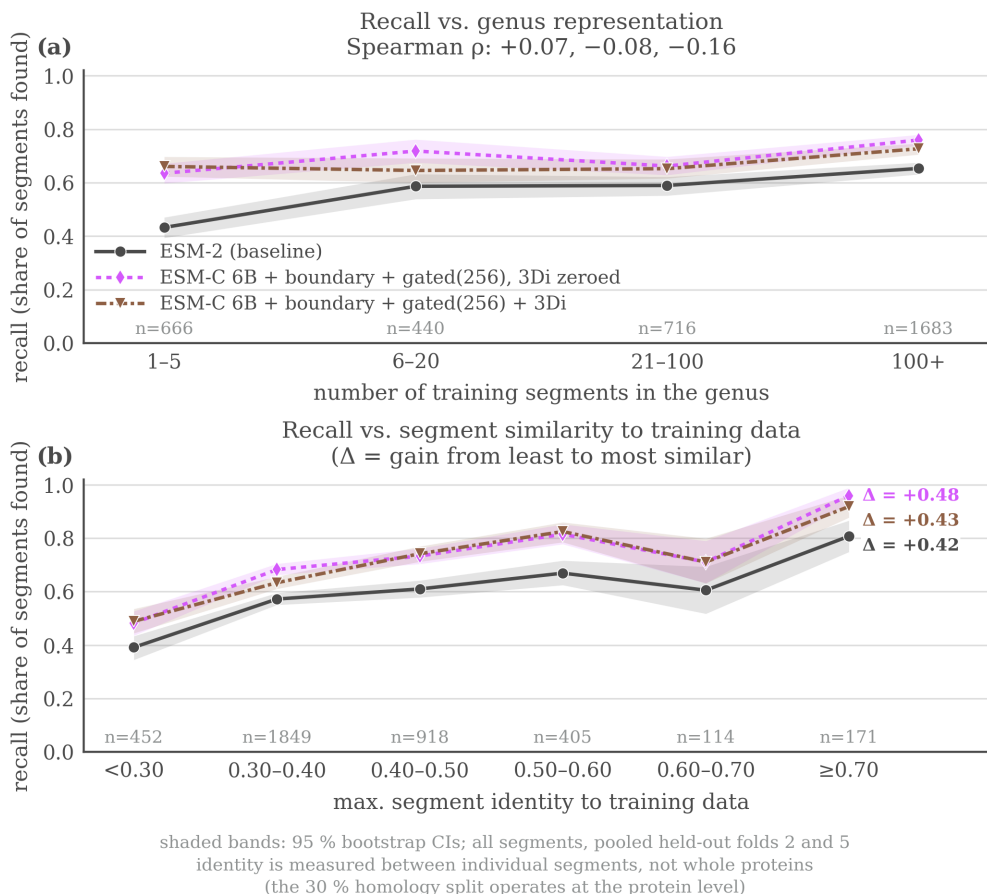


Figure 10: Recall on held-out peptides as a function of (a) how well-represented the protein's genus is in training and (b) maximum sequence identity to a training segment, screening protocol. For the two ESM-C models recall tracks maximum identity to a training segment far more than genus abundance, while for the ESM-2 baseline the two axes are closer to comparable. The x axis in (a) counts training *segments* in the genus, not proteins.

G Fold-level results under the confirmation protocol

The screening protocol of Appendix F raised a question it could not answer: how much of a measured difference belongs to the architecture and how much to the fold it was measured on. The confirmation protocol can answer it, because every configuration runs on all five outer folds and can be read fold by fold instead of only as an average. This section reports that reading. It is not further evidence for either addition, which Section 5 carries. It is the evidence behind the limitation stated in Section 6, that the folds are not exchangeable and that their spread is wider than the effects measured across them.

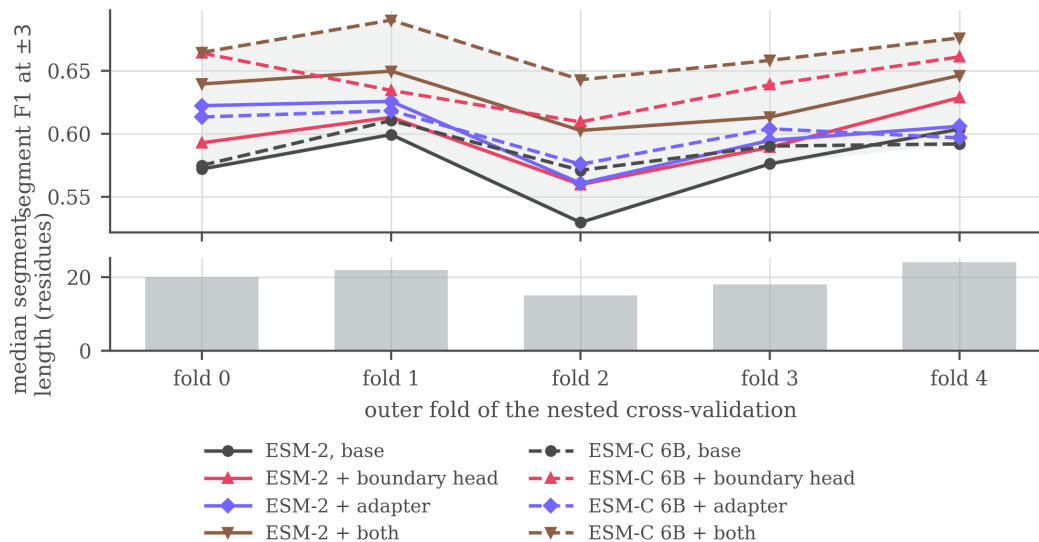


Figure 11: Segment F1 at ± 3 on each outer fold, every configuration on one axis, with the band between the best and worst configuration shaded, and below it the median length of the true segments each fold contains. The lines move together: fold 2 is the hardest for all eight and fold 1 or 4 the easiest for most, so much of the vertical movement within a line is the fold rather than the architecture. The folds were balanced on homology and cleavage motif, not on segment length, and that free axis tracks the difficulty: ordering them by median segment length and by the base architecture’s score differs in one adjacent swap. Within one configuration the score spans 0.040 to 0.074 across the folds, more than either addition is worth on its own, which is what makes a single split unable to resolve them and pairing by fold necessary. The ordering of configurations is largely preserved from fold to fold, which is why the paired comparisons of Section 5 are stable where the absolute levels are not.

Table 5: Segment F1 at ± 3 for every configuration on each of the five outer folds of the nested cross-validation, and in the last column the mean over the folds with their standard deviation. Each entry is the mean over the four inner models of that cell, scored with the corrected matcher.

Configuration	f0	f1	f2	f3	f4	mean
ESM-2, base	0.572	0.599	0.530	0.576	0.604	0.576 ± 0.029
ESM-2 + boundary head	0.593	0.613	0.560	0.589	0.629	0.597 ± 0.026
ESM-2 + adapter	0.622	0.626	0.561	0.595	0.606	0.602 ± 0.026
ESM-2 + both	0.639	0.650	0.603	0.613	0.646	0.630 ± 0.021
ESM-C 6B, base	0.575	0.611	0.571	0.590	0.592	0.588 ± 0.016
ESM-C 6B + boundary head	0.664	0.634	0.609	0.639	0.661	0.641 ± 0.022
ESM-C 6B + adapter	0.613	0.618	0.576	0.604	0.597	0.602 ± 0.017
ESM-C 6B + both	0.664	0.690	0.643	0.658	0.676	0.666 ± 0.018

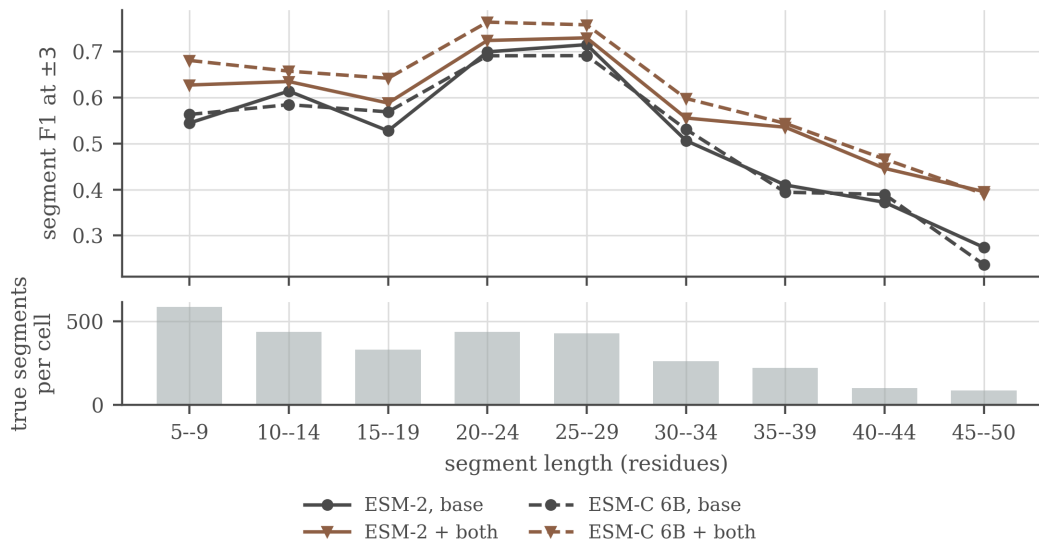


Figure 12: Why a fold’s length profile is not a cosmetic difference. Segment F1 at ± 3 against the length of the true segment, with the number of true segments in each bin below. Quality is far from flat in length: the base architecture scores 0.715 on segments of 25–29 residues and 0.274 on those of 45–50, a spread of 0.441, several times any effect this paper measures, and it falls away at the short end too. A fold’s length profile therefore sets how hard it is before any model touches it, and the five outer folds do differ on that axis, their median true segment running from 15 residues in fold 2 to 24 in fold 4 (Figure 11). That is what makes them non-exchangeable in the sense Section 6 means. The two additions help least where the base is already strongest, +0.015 at 25–29 residues, and most where it is weakest, +0.126 at 35–39 and +0.121 at 45–50, so they flatten the curve without levelling it.

540 H Boundary placement beyond the ± 3 window

541 Section 5 makes two claims that no single tolerance can settle: that the additions place boundaries
542 better rather than merely finding more segments, and that the split between them, placement alone
543 for the head and placement with coverage for the adapter, shows only once the acceptance window
544 is removed. This appendix reports the evidence for both.

545 **Holding detection fixed.** The comparison in Section 5 is paired. For each cell we take the true
546 segments that the variant and the base both localize within ± 3 , so both models have found the
547 segment and only its placement is at stake, and ask how often each puts a boundary on the annotated
548 residue. A configuration is twenty cells, five outer folds by four inner ones, and the four cells sharing
549 an outer fold are all tested on it, so each of the 15,348 true segments in the split is scored four times:
550 61,392 segment-instances in all, of which the base localizes 32,326. Pooled over the twenty cells
551 the paired set holds 27,013 instances for the boundary head, 29,277 for the adapter and 29,394 for
552 the two together. These are instances rather than distinct segments, so the paired comparison rests
553 on about a quarter as many independent segments as the counts suggest.

554 **What each block does to the count.** Those denominators are themselves the result. Of the 32,326
555 instances the base localizes, the boundary head localizes 320 fewer and the adapter 2,236 more, with
556 both together 3,289 more. About 1,400 of the adapter’s gain is coverage, its recall at a one-residue
557 gate being +0.024 over the base on these 61,392 instances, and the rest is segments the base already
558 overlapped but placed outside the window. The head therefore improves placement while covering
559 marginally less, which is the precision effect of Table 1 seen segment by segment, and the adapter
560 covers more while placing better still. On the paired set the head moves 13.2% of boundaries closer
561 to the annotated residue and 11.4% further away, a near-even trade that nets out positive, while the
562 two together move 15.0% closer against 7.5% further.

Dropping the window. A gate is itself a choice, so we also score the same predictions with no tolerance. Predicted and true segments of the same type are matched greedily one to one by intersection over union, and a match counts as a hit when the overlap clears a threshold on intersection over union. Precision and recall use all predicted and all true segments as their denominators, so nothing is conditioned on what either model happened to find. Every configuration is scored on the 35,504 protein-cells common to all runs, which takes the coverage difference between the two embeddings (8,897 against 8,999 proteins) out of the comparison. On that common set the ESM-2 to ESM-C 6B swap is worth +0.005 F1 at $\text{IoU} \geq 0.5$ under the base architecture, positive on two outer folds of five, and +0.024 under both additions, on five of five, reproducing the split that Section 5 measures at ± 3 .

Table 6: Segment F1 under overlap matching. The first column requires only that a predicted and a true segment of the same type share one residue, and the rest require an intersection over union of at least τ . Same cells as Table 1, restricted to the protein set common to every run, mean over the five outer folds.

	overlap	$\text{IoU} \geq 0.5$	$\text{IoU} \geq 0.8$	$\text{IoU} \geq 0.9$	$\text{IoU} \geq 0.95$
<i>ESM-2</i>					
base	0.729	0.667	0.569	0.487	0.411
+ boundary head	0.715	0.668	0.585	0.515	0.438
+ adapter	0.734	0.686	0.590	0.516	0.441
+ both	0.746	0.703	0.616	0.546	0.474
<i>ESM-C 6B</i>					
base	0.738	0.672	0.574	0.496	0.419
+ boundary head	0.735	0.704	0.624	0.555	0.478
+ adapter	0.729	0.681	0.587	0.514	0.440
+ both	0.759	0.727	0.649	0.582	0.507

Mostly placement. The looser the criterion, the less either addition is worth. Paired by outer fold against the ESM-2 base, at a gate that asks only for a single residue of overlap the boundary head is worth -0.013 ± 0.026 and the adapter $+0.006 \pm 0.012$, neither distinguishable from zero. At $\text{IoU} \geq 0.5$ they are worth $+0.001 \pm 0.021$ and $+0.019 \pm 0.017$, and at $\text{IoU} \geq 0.9$ $+0.028 \pm 0.004$ and $+0.028 \pm 0.015$, both on five folds of five. Underneath that near-zero F1 at the overlap gate the two move in opposite directions: the head issues 219 fewer segments per fold, buying $+0.022$ precision for -0.035 recall, while the adapter issues 130 more and buys $+0.024$ recall for -0.021 precision. The head finds no more of the annotation than the base does and the adapter only modestly more. What changes most is where the boundaries land, which is the conclusion Table 1 reaches through a tolerance, reached here without one.

The mean IoU taken over *all* true segments moves the other way for the head, 0.576 to 0.559, while its F1 rises at every threshold from 0.5 up. That is the precision effect of Table 1 seen from another angle: the head withdraws weak predictions, so some true segments lose a poor match altogether, and the ones that keep a match are placed better.

Which end moves. Scoring cleavage sites rather than segments, at exact placement and against all annotated sites, separates the two additions again. The boundary head improves the C-side of a segment, $+0.028 \pm 0.006$ on five folds of five, and leaves the N-side alone, -0.007 ± 0.020 and positive on only two of five. The adapter improves both, the N-side by $+0.022 \pm 0.006$ and the C-side by $+0.028 \pm 0.009$, and the two together by $+0.042 \pm 0.010$ and $+0.060 \pm 0.006$ respectively. Split by segment type, the head’s C-side gain is largest on propeptides ($+0.037 \pm 0.021$, five of five), whose C-terminal cut is where the mature peptide begins and is the weaker of the two C-sides: the base model scores 0.476 there against 0.525 on peptide C-sides.

Impact Statement

This paper improves a computational tool for predicting proteolytic cleavage products from protein sequence. Potential applications include interpreting proteomics data, informing vaccine and peptide-drug design, and studying disease-associated proteolysis. We are not aware of societal risks

599 specific to this work beyond those generally associated with more accurate protein sequence analysis
600 tools.